

Code Logic Review Report

our-sanctuary-gemini-main

A Couples Private Sanctuary App (React + Capacitor + Supabase)

Prepared by: Z.ai Code Logic Reviewer

Date: 2026-05-22

Category	Count
Critical (Security / Data Loss)	6
High (Logic Errors / Race Conditions)	9
Medium (Incorrect Behavior / UX Bugs)	7
Low (Code Smell / Minor Issues)	4
Total Findings	26

Table of Contents

1. Executive Summary	4
2. Critical Findings (Security / Data Loss)	4
2.1 Hardcoded Supabase Credentials in Client Bundle	4
2.2 Trivially Bypassable Row Level Security Policies	4
2.3 Firebase Dependency Will Crash at Runtime	5
2.4 Race Condition in supabaseFirestoreCompat setDoc	5
2.5 Encryption Stores Both Ciphertext and Plaintext	6
2.6 Message ID Collision Between App and WhatsApp Imports	6
3. High Severity Findings (Logic Errors / Race Conditions)	7
3.1 onSnapshot Deduplication Bug Can Suppress Legitimate Updates	7
3.2 Realtime Payload Structure Mismatch	7
3.3 Infinite Re-render Loop in ChatScreen "Seen" Status	7
3.4 Presence Update Race Condition	8
3.5 Typing Status Race Condition	8
3.6 Network Detection Listener Leak	8
3.7 Game Session Auto-Creation Side Effect in onSnapshot	8
3.8 Missing Return from onForegroundMessage	9
3.9 session.cookie.secure=true Breaks HTTP Development	9
4. Medium Severity Findings (Incorrect Behavior / UX Bugs)	9
4.1 Error Message References "Firebase" After Migration	9
4.2 Backup Restore Does Not Sync to Cloud	9
4.3 Date Parsing in WhatsApp Import Assumes Specific Format	10
4.4 Search Only Filters Deleted Messages, Not Imported	10
4.5 PWA Manifest Name is "512" Instead of App Name	10
4.6 Suboptimal setDoc Usage Across Multiple Screens	10
4.7 Vite Define Injects GEMINI_API_KEY Into Client Bundle	11
5. Low Severity Findings (Code Smell / Minor Issues)	11
5.1 Unused firebase.ts File	11
5.2 Duplicate Notification Logic in notifications.ts	11

5.3 Hardcoded Partner Names in ChatScreen Import Logic	11
5.4 Background Upload Button Does the Same as Profile Upload	12
6. Findings Summary Table	12
7. Recommendations	13
7.1 Immediate Actions (Before Any Deployment)	13
7.2 Short-Term Fixes (Next Sprint)	13
7.3 Medium-Term Improvements	14

1. Executive Summary

This report documents the results of a comprehensive code-logic review of the **our-sanctuary-gemini** project, a React/TypeScript couples sanctuary application that was recently migrated from Firebase to Supabase. The review focused on logical correctness, race conditions, security vulnerabilities, data integrity issues, and migration-related bugs. A total of **26 findings** were identified across 6 severity levels, including 6 critical issues that could lead to data loss, unauthorized access, or application crashes.

The most severe findings include: (1) hardcoded Supabase credentials in client-side code, (2) trivially bypassable Row Level Security policies, (3) a stale Firebase dependency that will crash at runtime, (4) a race condition in the Firestore-compat layer that causes data overwrites on concurrent writes, (5) encryption that stores both ciphertext and plaintext in the same localStorage key, and (6) message ID collisions between app messages and imported WhatsApp messages. These issues should be addressed before any production deployment.

2. Critical Findings (Security / Data Loss)

2.1 Hardcoded Supabase Credentials in Client Bundle

File: src/lib/supabase.ts (lines 3-4)

Severity: CRITICAL

Impact: Anyone who inspects the bundled JavaScript can extract the Supabase URL and anon key. While the anon key is designed to be public, the hardcoded fallback URL means attackers can directly query the Supabase instance, especially given the weak RLS policies (see 2.2). The MIGRATION_CHANGES.md also notes that the service_role key was previously exposed and needs rotation.

```
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
|| 'https://fqtooqlafypnhryfnpfo.supabase.co';
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY
|| 'sb_publishable_gkXzmAefCJMmSctPpuAXZg_IddPU2yq';
```

Fix: Remove the hardcoded fallback values. If environment variables are missing, the app should display a configuration error rather than silently connecting to a specific instance. Rotate the Supabase anon key and verify that the previously-exposed service_role key has been rotated.

2.2 Trivially Bypassable Row Level Security Policies

File: supabase-setup.sql (lines 27-39)

Severity: CRITICAL

Impact: The RLS policies grant full CRUD access to any row where the primary key

length is at least 6 characters. Since vault codes are always 6-character strings, any client with the anon key can read, modify, or delete any couple's data. There is no authentication requirement whatsoever.

```
create policy "couple_state app access"
on public.couple_state for all
using (length(vault_id) >= 6)
with check (length(vault_id) >= 6);
```

Fix: Implement proper authentication (Supabase Auth) and tie RLS policies to authenticated user identities. At minimum, use a vault-specific secret that is not the vault_id itself. The current design relies entirely on security through obscurity (the 6-char vault code).

2.3 Firebase Dependency Will Crash at Runtime

File: src/lib/fcm.ts (lines 1-2)

Severity: CRITICAL

Impact: The file imports from firebase/app and firebase/messaging, but Firebase packages were removed from package.json during the Supabase migration. This means the import will throw a module-not-found error at runtime. The initFCM() function is called unconditionally in App.tsx (line 130), so the app will crash on startup unless the try/catch in initFCM somehow prevents the import error from propagating - but dynamic imports are not used here, so the crash is guaranteed.

```
import { initializeApp } from 'firebase/app'
import { getMessaging, getToken, onMessage } from 'firebase/messaging'
```

Fix: Either (a) re-add Firebase as a dependency if FCM push notifications are still needed, or (b) replace fcm.ts with a Supabase-native push notification solution and remove the Firebase imports entirely. The MIGRATION_CHANGES.md already notes that remote push notifications require backend infrastructure, so option (b) is recommended.

2.4 Race Condition in supabaseFirestoreCompat setDoc

File: src/lib/supabaseFirestoreCompat.ts (lines 81-97)

Severity: CRITICAL

Impact: When { merge: true } is used, setDoc first reads the current row, deep-merges the updates, then upserts. If two clients call setDoc concurrently (e.g., both partners sending messages simultaneously), both read the same base state, each merges their own changes independently, and the second upsert overwrites the first client's changes. This is a classic read-modify-write race condition that will cause silent data loss.

```
export async function setDoc(ref, updates, options) {
  const currentSnap = options?.merge ? await getDoc(ref) : null;
  const currentData = currentSnap?.exists() ? currentSnap.data() : {};
  const nextData = options?.merge ? deepMerge(currentData, updates) : updates;
```

```
// ... upsert(nextData) -- last writer wins!
```

Fix: Use Supabase RPC functions (Postgres functions) for atomic merge operations, or implement optimistic concurrency control using a version/timestamp column. Alternatively, restructure the data model so that each update targets a separate row (e.g., individual message rows instead of a single JSON blob containing all messages).

2.5 Encryption Stores Both Ciphertext and Plaintext

File: src/App.tsx (lines 219-232)

Severity: CRITICAL

Impact: When encryption is enabled, the local persistence logic encrypts the state but sets `messages: []` in the encrypted payload. However, the object spread `{ ...state, messages: [], isEncrypted: true, encryptedPayload: encryptData({ ...state, messages: [] }, key) }` still includes all other state properties (`batmanName`, `princessName`, `moods`, `events`, etc.) in plaintext alongside the encrypted payload. An attacker with access to `localStorage` gets both the plaintext fields and the encrypted blob, severely undermining the purpose of encryption.

```
let storageData: any = state;
if (state.encryptionKey) {
  storageData = {
    ...state, // All fields still in plaintext!
    messages: [],
    isEncrypted: true,
    encryptedPayload: encryptData({ ...state, messages: [] }, key)
  };
}
```

Fix: When encryption is enabled, store ONLY the encrypted payload and the `isEncrypted` flag. Do not spread the full state object. The correct structure should be: `{ isEncrypted: true, encryptedPayload: encryptData(state, key) }`. The same issue applies to the cloud sync path (line 247) where decrypted data is spread alongside encrypted data.

2.6 Message ID Collision Between App and WhatsApp Imports

File: src/screens/ChatScreen.tsx (lines 329-342)

Severity: CRITICAL

Impact: App-generated messages use `Date.now() + Math.floor(Math.random() * 1000)` as IDs, while imported WhatsApp messages use Unix timestamps as IDs. Since `Date.now()` returns a Unix timestamp in milliseconds, there is a high probability of ID collisions. When merging messages in the Map, later entries overwrite earlier ones with the same ID, causing messages to silently disappear.

```
// App messages:
id: Date.now() + Math.floor(Math.random() * 1000)
```

```
// Imported messages:
id: ts // where ts = new Date(...).getTime()
```

Fix: Use a more robust ID generation strategy. For example, prefix imported messages with "wa-" or use UUIDs. Alternatively, use separate ID namespaces so that app messages and imported messages never collide in the merge Map.

3. High Severity Findings (Logic Errors / Race Conditions)

3.1 onSnapshot Deduplication Bug Can Suppress Legitimate Updates

File: src/lib/supabaseFirestoreCompat.ts (lines 103-152)

Severity: HIGH

Impact: The onSnapshot function compares the initial data hash with the realtime event hash to skip the duplicate initial-load event. However, if a different client updates the data between the initial fetch and the realtime subscription becoming active, the realtime event will carry the same data as the initial fetch (which is now stale). The hash comparison will match, and the update will be silently dropped. Additionally, after the first realtime event, initialDataHash is set to null, so the dedup logic only works for the very first event and cannot prevent duplicates from reconnections.

3.2 Realtime Payload Structure Mismatch

File: src/lib/supabaseFirestoreCompat.ts (lines 130-139)

Severity: HIGH

Impact: The makeSnapshot function assumes that the data is either wrapped in a .data property or is the raw object. For the initial load via getDoc, the Supabase row returns { vault_id, data, updated_at }, and makeSnapshot correctly extracts .data. However, the realtime payload's payload.new also has the same structure { vault_id, data, updated_at }, but it is passed directly to makeSnapshot which will call .data() on it, returning the nested JSON. This means the realtime snapshot structure differs from the initial snapshot, causing inconsistent data access patterns in the consumer.

3.3 Infinite Re-render Loop in ChatScreen "Seen" Status

File: src/screens/ChatScreen.tsx (lines 394-409)

Severity: HIGH

Impact: The useEffect for marking messages as "seen" depends on [selectedChat, state.messages]. When unseen messages are found, it calls setState to update the message status, which changes state.messages, triggering the effect again. The

seenMessageIds ref prevents re-processing the same messages, but the effect still runs on every state change, causing unnecessary re-renders and redundant syncMessages calls. In the worst case, if syncMessages triggers a cloud update that produces a new snapshot, this can create a feedback loop.

3.4 Presence Update Race Condition

File: src/App.tsx (lines 358-447)

Severity: HIGH

Impact: The presence update reads the current stateRef.current.presenceStatus, modifies it locally, then writes it back via setDoc. Since setDoc({ merge: true }) uses the read-modify-write pattern (see finding 2.4), two concurrent presence updates will overwrite each other. This means one partner's online status can be lost when both update simultaneously.

3.5 Typing Status Race Condition

File: src/screens/ChatScreen.tsx (lines 466-491)

Severity: HIGH

Impact: Same race condition as presence updates. The typing status is read from local state, modified, then written back via setDoc({ merge: true }). Concurrent updates from both partners will cause one partner's typing status to be lost.

3.6 Network Detection Listener Leak

File: src/lib/networkDetection.ts (lines 14-31)

Severity: HIGH

Impact: The initNetworkDetection() function adds "online" and "offline" event listeners to the window but never returns an unsubscribe function. Each call to initNetworkDetection() adds new listeners without removing the previous ones. In React strict mode (which is enabled in main.tsx), effects run twice during development, causing duplicate listeners. The event listeners are never cleaned up on component unmount, leading to memory leaks and duplicate callback invocations.

3.7 Game Session Auto-Creation Side Effect in onSnapshot

File: src/screens/SanctuaryScreen.tsx (lines 118-132)

Severity: HIGH

Impact: The onSnapshot callback for game sessions calls setDoc to create a new session when the snapshot doesn't exist. This write triggers another realtime event, which calls the snapshot handler again, potentially in an infinite loop. While the second call would find the document exists, it still represents a side effect inside a read callback that can cause unexpected behavior, especially if the initial write fails or is delayed.

3.8 Missing Return from onForegroundMessage

File: src/lib/fcm.ts (line 67)

Severity: HIGH

Impact: The onForegroundMessage function returns () => {} (a no-op) when messaging is not initialized, but in App.tsx (line 185) the return value is used as the cleanup function for a useEffect. If messaging fails to initialize (which is likely given finding 2.3), the useEffect will store this no-op unsubscribe. The actual onMessage listener from Firebase (if it were working) would never be cleaned up, causing a listener leak.

3.9 session.cookie.secure=true Breaks HTTP Development

File: server.ts (line 47)

Severity: HIGH

Impact: The session cookie is configured with secure: true, which means cookies will only be sent over HTTPS. During local development with npm run dev (which serves HTTP on port 3000), the session cookie will never be sent, making the session middleware non-functional. This is especially problematic since the README instructs users to run locally via HTTP.

4. Medium Severity Findings (Incorrect Behavior / UX Bugs)

4.1 Error Message References "Firebase" After Migration

File: src/lib/media.ts (lines 28-30)

Severity: MEDIUM

Impact: The error messages still reference "Firebase Storage" even though the project has been migrated to Supabase. The error message reads: "Media upload failed. Enable Firebase Storage and deploy storage.rules." This is confusing for developers and users who are now using Supabase, not Firebase. Similarly, line 27 logs "Firebase Storage upload failed."

4.2 Backup Restore Does Not Sync to Cloud

File: src/lib/backup.ts (lines 110-122)

Severity: MEDIUM

Impact: The restoreBackupToState function returns a partial state object but does not sync the restored data to the cloud. After importing a backup, the data only exists locally. If the user expects the restore to also update the cloud copy (which is a reasonable expectation), they will lose the restored data when the next cloud sync overwrites the local state with the older cloud version.

4.3 Date Parsing in WhatsApp Import Assumes Specific Format

File: `src/screens/ChatScreen.tsx` (lines 232-242)

Severity: MEDIUM

Impact: The `parseMsg` function assumes the WhatsApp date format is `DD/MM/YYYY` and the time format uses 12-hour `AM/PM` notation. WhatsApp exports from different locales or regions may use different date/time formats (e.g., `MM/DD/YYYY`, 24-hour time). Messages with unrecognized formats are silently dropped, leading to incomplete chat imports with no user feedback about skipped messages.

4.4 Search Only Filters Deleted Messages, Not Imported

File: `src/App.tsx` (line 687)

Severity: MEDIUM

Impact: The search modal filters messages with `!m.deleted` but does not check `!m.imported`. This means imported WhatsApp messages will appear in search results, which may be unexpected. While not strictly a bug, it creates an inconsistency: the search shows imported messages but they are not part of the "real" message state.

4.5 PWA Manifest Name is "512" Instead of App Name

File: `vite.config.ts` (lines 17-18)

Severity: MEDIUM

Impact: The PWA manifest has `name: "512"` and `short_name: "512"`, which appears to be a placeholder (likely the days-together count) instead of the actual app name. When users install the PWA, it will appear as "512" on their home screen, which is confusing. The `theme_color` and `background_color` are also set to `#0F172A` (dark navy) which does not match any of the app's themes.

4.6 Suboptimal `setDoc` Usage Across Multiple Screens

Files: `HomeScreen.tsx`, `SanctuaryScreen.tsx`, `MemoriesScreen.tsx`, `SettingsScreen.tsx`

Severity: MEDIUM

Impact: Multiple screens call `setDoc(doc(db, "couples", vaultId), { aiMemory: { ...state.aiMemory, explicitMemories: updatedMemories } }, { merge: true })`. Because `merge: true` does a deep merge, the partial `aiMemory` object will be deep-merged with whatever is currently in the database row. This means if the local `state.aiMemory` is stale (due to a missed realtime update), the merge will overwrite the cloud's version of nested fields like `chosenInteractions` with stale local values, causing data regression.

4.7 Vite Define Injects GEMINI_API_KEY Into Client Bundle

File: vite.config.ts (lines 50-52)

Severity: MEDIUM

Impact: The Vite config defines `process.env.GEMINI_API_KEY` as a compile-time replacement, which means the API key will be embedded in the client-side JavaScript bundle. Anyone who inspects the bundle can extract the key. The app also stores the API key in `localStorage` (`SettingsScreen`), but the `server.ts` uses a separate `OPENROUTER_API_KEY` environment variable. The `GEMINI_API_KEY` client-side injection serves no purpose since the AI calls go through the Supabase Edge Function or server, not directly from the client.

5. Low Severity Findings (Code Smell / Minor Issues)

5.1 Unused firebase.ts File

File: src/lib/firebase.ts

Severity: LOW

Impact: This file contains only `export const db = {};` and is a leftover from the Firebase migration. It is not imported anywhere in the current codebase. While harmless, it adds confusion and should be removed.

5.2 Duplicate Notification Logic in notifications.ts

File: src/lib/notifications.ts

Severity: LOW

Impact: The `showSystemNotification` function in `notifications.ts` duplicates most of the logic in `notificationService.ts`'s `showLocalNotification`. They both handle Capacitor and web notification display, but with slightly different parameters (e.g., different vibration durations: 500ms vs 300ms). This creates maintenance burden and inconsistent notification behavior.

5.3 Hardcoded Partner Names in ChatScreen Import Logic

File: src/screens/ChatScreen.tsx (lines 326-327)

Severity: LOW

Impact: The WhatsApp import logic hardcodes `obadaName = "Obada Shabanieh"` and `wifeName = "My Lovely Wife"` as the sender name mapping. This makes the import feature only work for one specific couple's chat export. The `ImportedChatViewer` component also hardcodes "Obada Shabanieh" as the sender check (line 67). This

should be configurable or derived from the app's identity settings.

5.4 Background Upload Button Does the Same as Profile Upload

File: src/screens/SettingsScreen.tsx (lines 506-519)

Severity: LOW

Impact: In the ProfileCard component, both the "Bg" button and "Profile" button call `onPhotoChange(url)`, which updates the profile photo. The "Bg" (background) button should update a different field (e.g., a background photo), but it calls the same handler. This is likely a copy-paste bug where the background upload handler was not properly differentiated from the profile upload handler.

6. Findings Summary Table

#	Finding	Severity	File
2.1	Hardcoded Supabase Credentials	CRITICAL	supabase.ts
2.2	Bypassable RLS Policies	CRITICAL	supabase-setup.sql
2.3	Firebase Import Will Crash	CRITICAL	fcm.ts
2.4	setDoc Race Condition	CRITICAL	supabaseFirestoreCompat.ts
2.5	Encryption Plaintext Leak	CRITICAL	App.tsx
2.6	Message ID Collisions	CRITICAL	ChatScreen.tsx
3.1	onSnapshot Dedup Bug	HIGH	supabaseFirestoreCompat.ts
3.2	Realtime Payload Mismatch	HIGH	supabaseFirestoreCompat.ts
3.3	Infinite Re-render in Seen Status	HIGH	ChatScreen.tsx
3.4	Presence Update Race	HIGH	App.tsx
3.5	Typing Status Race	HIGH	ChatScreen.tsx
3.6	Network Listener Leak	HIGH	networkDetection.ts
3.7	Game Session Auto-Create Loop	HIGH	SanctuaryScreen.tsx
3.8	Missing Cleanup in FCM	HIGH	fcm.ts
3.9	Session Cookie Secure Flag	HIGH	server.ts
4.1	Firebase Error Messages	MEDIUM	media.ts
4.2	Backup No Cloud Sync	MEDIUM	backup.ts
4.3	WhatsApp Date Parsing	MEDIUM	ChatScreen.tsx

4.4	Search Shows Imported	MEDIUM	App.tsx
4.5	PWA Manifest Name Wrong	MEDIUM	vite.config.ts
4.6	Stale Deep Merge Overwrites	MEDIUM	Multiple files
4.7	API Key in Client Bundle	MEDIUM	vite.config.ts
5.1	Unused firebase.ts	LOW	firebase.ts
5.2	Duplicate Notification Logic	LOW	notifications.ts
5.3	Hardcoded Partner Names	LOW	ChatScreen.tsx
5.4	Bg Button Same as Profile	LOW	SettingsScreen.tsx

7. Recommendations

7.1 Immediate Actions (Before Any Deployment)

- 1. Remove hardcoded credentials** from supabase.ts. Use environment variables exclusively and fail gracefully when they are not configured. Rotate all exposed keys including the Supabase anon key and the previously-exposed service_role key.
- 2. Fix or remove the Firebase dependency** in fcm.ts. Since Firebase packages are not in package.json, the app will crash on startup. Either add Firebase back as a dependency or replace FCM with a Supabase-native push notification solution.
- 3. Fix the encryption logic** so that when encryption is enabled, only the encrypted payload is stored in localStorage and synced to the cloud. Never store plaintext state alongside the encrypted payload.
- 4. Implement proper RLS policies** that require authentication. The current policies that only check key length provide zero security against anyone with the anon key.

7.2 Short-Term Fixes (Next Sprint)

- 5. Fix the read-modify-write race condition** in supabaseFirestoreCompat.ts by either (a) using Supabase RPC functions for atomic operations, (b) implementing optimistic concurrency control with version columns, or (c) restructuring the data model to use individual rows instead of a single JSON blob.
- 6. Fix message ID collisions** by using UUIDs or namespaced IDs that distinguish between app-generated messages and imported WhatsApp messages.
- 7. Fix the network detection listener leak** by returning an unsubscribe function from initNetworkDetection() and calling it in the useEffect cleanup.
- 8. Fix the realtime payload structure mismatch** so that both the initial load and realtime events produce consistent snapshot structures. The payload.new should be

unwrapped the same way as the `getDoc` result.

7.3 Medium-Term Improvements

9. Refactor the data model from a single JSON blob per couple to a normalized schema with separate tables for messages, memories, events, letters, etc. This eliminates the race condition problems inherent in read-modify-write patterns on a shared JSON blob and enables proper Supabase realtime subscriptions on granular data changes.

10. Add comprehensive error handling across the app. Many async operations catch errors with `console.error` only, without informing the user. Failed syncs, failed uploads, and failed real-time connections should surface meaningful errors to the user, not just log to the console.

11. Remove all Firebase references including the stale `fcms.ts`, the unused `firebase.ts`, error messages mentioning "Firebase Storage", and the Vite alias that redirects `firebase/firestore` to the compat layer. The migration is incomplete and these remnants will confuse future developers.

12. Add unit tests for the critical `supabaseFirestoreCompat` layer, the encryption logic, and the message merge logic. These are the most bug-prone areas and currently have zero test coverage.